

BANANA INSTANCE SEGMENTATION AND HEALTH CLASSIFICATION

Using YOLO11m-seg and the Official Ultralytics Framework Source

Complete Project Documentation, Evaluation Report,
Google Drive Review Guide, and Reproduction Instructions

Author	Huzaifa Waqar Butt
Project status	Finalized and ready for academic review
Final model	YOLO11m-seg
Documentation date	6 August 2026
Primary framework	Ultralytics 8.4.115
Storage location	Google Drive: Segmentation_Project

Final-model statement: The completed project uses the standard Ultralytics YOLO11m-seg training and evaluation pipeline. The final Google Drive workspace contains the cleaned scripts, final checkpoints, standard evaluation evidence, logs, visualizations, and centralized results.

Contents

1. Executive Summary
 2. Project Background, Scope, and Objectives
 3. Framework Source and Model Selection
 4. Development Environment
 5. Final Google Drive Directory Structure
 6. Dataset Description and Preparation
 7. End-to-End Project Workflow
 8. Training Configuration and Implementation
 9. Evaluation Methodology
 10. Final Evaluation Results
 11. Final Model, Results, and Log Locations
 12. Using the Trained Model
 13. Sharing and Reviewing the Project through Google Drive
 14. Reproducing Training and Evaluation
 15. Limitations and Responsible Interpretation
 16. Final Project Closure
- Appendix A. Key Files and Reports
- Appendix B. Teacher Review Checklist
- References

1. Executive Summary

This project, completed by Huzaifa Waqar Butt, developed a two-class banana instance-segmentation system that detects each banana, separates its pixel-level mask from the background, and classifies it as either healthy_banana or unhealthy_banana. The final implementation used the official Ultralytics framework source and the pretrained YOLO11m-seg model.

The work was completed in Google Colab with Google Drive persistence. The project included structured storage, environment verification, dataset auditing and preparation, deterministic training configuration, managed experiment execution, checkpoint handling, full training, and final test-only evaluation. The final model completed 100 epochs in approximately 63.30 minutes on an NVIDIA A100-SXM4-80GB GPU.

On the final research test split of 359 images and 2,513 labelled banana instances, the model achieved 99.994% precision, 100% recall, and 99.5% mAP@50 for both bounding boxes and segmentation masks. The stricter mAP@50-95 was 91.19% for bounding boxes and 78.96% for segmentation masks.

Interpretation: The value 99.5% is mAP@50, not a universal real-world accuracy guarantee. The dataset was derived from only eight original source images, so performance must be validated on genuinely new images before broad deployment claims are made.

2. Project Background, Scope, and Objectives

2.1 Problem statement

Banana inspection requires more than image-level classification. A practical image can contain several bananas, overlapping objects, and complex backgrounds. The system must therefore identify each individual banana, classify its health condition, and estimate its exact shape. Instance segmentation is appropriate because it combines object detection, class prediction, and a separate pixel-level mask for every detected object.

2.2 Primary objectives

- Prepare and verify a two-class YOLO instance-segmentation dataset.
- Train a reliable pretrained segmentation model using a structured and reproducible pipeline.
- Save final checkpoints, metadata, logs, plots, and metrics in Google Drive.
- Evaluate the best checkpoint on a test split not used for model parameter optimization.
- Provide clear evidence that a reviewer can inspect and re-evaluate.
- Document the complete workflow for academic review and future continuation.

2.3 Final project scope

The final deliverable is a standard YOLO11m-seg banana instance-segmentation model, accompanied by the complete cleaned scripts, research and benchmark datasets, final training evidence, standard test metrics, logs, and visualizations. The project is finalized in its original Google Drive workspace and is ready to be shared with a teacher using Drive permissions.

3. Framework Source and Model Selection

3.1 Official framework source used

Item	Final selection	Reason
Framework source	Official Ultralytics	Provides maintained YOLO instance-segmentation support, a clear Python API, integrated training and validation, and standard output artifacts.
Official source URL	https://github.com/ultralytics/ultralytics	Identifies the upstream framework used by the project; the user project itself is stored in Google Drive.
Local version	Ultralytics 8.4.115	The exact runtime version verified before training and evaluation.
Verified commit	8d7870201be0edf01393ba73f3a632248e31b195	Records the exact local source revision used.
Local framework path	/content/drive/MyDrive/Segmentation_Project/02_Repositories/ultralytics	The official framework source is preserved inside the final Google Drive project.

The final project uses the local official Ultralytics source included in Google Drive. This allows the teacher to inspect the exact framework files used by the training pipeline without requiring a separate user-project repository.

3.2 Final model used

Property	Value
Model	YOLO11m-seg
Task	Instance segmentation
Pretraining	Official Ultralytics pretrained weights
Pretrained checkpoint	/content/drive/MyDrive/Segmentation_Project/03_Pretrained_Weights/yolo11m-seg.pt
Approximate fused parameters	22,336,854
Evaluation architecture	139 fused layers
Classes	2: healthy_banana and unhealthy_banana

YOLO11m-seg was selected as a medium-capacity model. It provides more representational capacity than nano and small variants while remaining more practical than the large and extra-large variants. It supports bounding boxes, class predictions, and instance masks within one end-to-end architecture and was suitable for the available A100 GPU.

4. Development Environment

Component	Verified environment
Platform	Google Colab with Google Drive mounted
Python	3.12.13
PyTorch	2.11.0+cu128
CUDA	12.8 runtime
GPU	NVIDIA A100-SXM4-80GB
Ultralytics	8.4.115 from the local official framework source
Input image size	640 x 640
Random seed	42
Deterministic mode	Enabled
Mixed precision	AMP enabled

5. Final Google Drive Directory Structure

After project cleanup and safe folder renumbering, the final workspace is organized under:

```
/content/drive/MyDrive/Segmentation_Project
```

```
Segmentation_Project/
|-- 01_Datasets/
|   |-- dataset/           # Original/raw dataset
|   |-- Prepared_Dataset/
|   |   |-- research/      # Primary augmentation-family split
|   |   |-- benchmark/     # Secondary benchmark split
|-- 02_Repositories/
|   |-- ultralytics/       # Local official framework source
|-- 03_Pretrained_Weights/
|   |-- yolo11m-seg.pt
|-- 04_Training_Runs/
```

```

|-- banana_segmentation_full_v1_20260805_184146/
|   |-- weights/           # best.pt and last.pt
|   |-- training/          # results.csv, args.yaml, plots
|   |-- metadata/          # configurations and test report
|   |-- test_evaluation_20260805_195131/
|-- 05_Logs/               # Final logs and verification reports
|-- 06_Scripts/            # Final executable project scripts
|-- 07_Results/
|   |-- 01_Final_Training/
|   |-- 02_Final_Test_Evaluation/
|   |-- final_metrics.json
|-- 08_Visualizations/     # Dataset and result visualizations

```

Final structure: The numbering is sequential and the scripts were updated to use the renamed paths. The final best and last model hashes were verified before and after renumbering.

6. Dataset Description and Preparation

6.1 Dataset composition

Dataset element	Count / description
Original source images	8
Augmented image-label pairs	2,872
Total raw image-label pairs	2,880
Annotation format	YOLO instance-segmentation polygons
Class 0	healthy_banana
Class 1	unhealthy_banana

Every label line begins with a class identifier followed by normalized polygon coordinates. The project verified image-label pairing, class identifiers, polygon structure, coordinate ranges, and split integrity. The raw data were preserved. Prepared copies clipped invalid normalized polygon coordinates into the valid $[0, 1]$ range. In total, 632 raw label files contained at least one out-of-range coordinate before preparation.

6.2 Primary research split

Split	Images	Purpose
Train	1,795	Model parameter optimization
Validation	718	Model selection and training monitoring
Test	359	Final test-only evaluation
Total	2,872	Prepared augmented examples used in the research split

The research split was constructed by augmentation family rather than by randomly separating individual augmented images. Five original-image families were assigned to training, two to validation, and one to testing. This reduced direct leakage between augmentations of the same source image and made the research split more defensible than a purely random image-level split.

6.3 Secondary benchmark split

Split	Images
Train	2,297
Validation	287
Test	288

The benchmark split was retained as a secondary conventional comparison. The research split was used for the reported final model because it better separated augmentation families.

Dataset limitation: Although thousands of augmented samples were available, they originated from only eight independent source images. Augmentation increases variation but does not replace collecting new cameras, backgrounds, lighting conditions, banana varieties, maturity stages, and disease appearances.

7. End-to-End Project Workflow

Stage	What was completed
Project initialization	Created the original Google Drive workspace and a professional directory layout.
Framework setup	Stored and verified the official Ultralytics framework source, version, and commit.
Environment verification	Confirmed Python, PyTorch, CUDA, GPU availability, local import precedence, and pretrained weights.
Dataset audit	Checked image-label pairs, polygon structure, coordinate ranges, class IDs, corrupt files, and counts.
Dataset preparation	Created prepared research and benchmark splits while preserving the original dataset.
Configuration system	Implemented a validated TrainingConfig dataclass with JSON export and reproducibility settings.
Experiment management	Implemented managed run directories, metadata, logging, checkpoint hashes, status tracking, and resume support.
Pipeline verification	Ran a limited sanity execution before the final full training run.
Full training	Trained YOLO11m-seg on the complete research training split for 100 epochs.
Test evaluation	Loaded best.pt and evaluated it on the 359-image research test split without further training.
Project cleanup	Removed incomplete runs, caches, placeholders, duplicate intermediate artifacts, and unused development files.
Finalization	Centralized results, verified model hashes, renumbered folders, updated script paths, and prepared the project for review.

8. Training Configuration and Implementation

8.1 Final training configuration

Parameter	Final value	Purpose
Experiment name	banana_segmentation_full_v1	Identifies the managed final run.
Dataset	research	Primary augmentation-family-separated split.
Model	yolo11m-seg	Medium-capacity instance-segmentation model.
Epochs	100	Completed full training duration.
Batch size	16	Balanced throughput and GPU memory use.
Image size	640	Training and evaluation resolution.
Workers	2	Stable Google Drive-backed data loading.
Device	CUDA device 0	NVIDIA A100 GPU.
Optimizer	auto	Ultralytics-managed optimizer selection.
AMP	True	Mixed-precision acceleration.
Deterministic	True	Improved repeatability.

Parameter	Final value	Purpose
Seed	42	Controlled random operations.
Patience	50	Early-stopping monitoring setting.
Checkpoint interval	Every 10 epochs	Periodic recovery checkpoints during training.
Cache	False	Avoided unnecessary Drive storage pressure.

The full run completed without resuming from an interrupted checkpoint. Total elapsed time was 3,797.74 seconds, or approximately 63.30 minutes. The final training CSV contains 100 epoch rows, and Ultralytics validated the selected best checkpoint after training.

8.2 Final project scripts

Script	Responsibility
initialize_project.py	Creates and verifies the project directory structure.
bootstrap.py	Prepares Python import paths and project startup behavior.
config.py	Defines final project paths, classes, dataset locations, framework source, weights, runs, logs, results, and visualizations.
prepare_dataset.py	Creates and verifies prepared dataset copies and split structures.
verify_environment.py	Checks Python, GPU, dependencies, local source, and required project paths.
verify_dataset.py	Validates the selected prepared dataset and image-label integrity.
training_config.py	Defines validated training settings, JSON export, and reproducibility controls.
model_factory.py	Loads supported YOLO11 segmentation variants from local pretrained weights.
experiment_manager.py	Manages the final run, metadata, status, hashes, artifacts, and summaries.
trainer.py	Coordinates verification, training, resume logic, metrics, and checkpoint collection.
logger.py	Writes structured project and training logs.
utils.py	Provides timestamps, JSON persistence, hashing, and shared helper functions.

9. Evaluation Methodology

9.1 Standard Ultralytics test evaluation

The final best.pt checkpoint was loaded and evaluated with `model.val(split="test")` using the research dataset YAML. This stage did not update the model weights. The test set contained 359 images and 2,513 annotated instances: 1,795 healthy bananas and 718 unhealthy bananas.

The standard evaluation reports precision, recall, mAP@50, and mAP@50-95 for both bounding boxes and masks. mAP@50 measures performance at an intersection-over-union threshold of 0.50. mAP@50-95 averages performance across stricter thresholds from 0.50 through 0.95 and is therefore a stronger measure of localization and segmentation quality.

9.2 Evidence verification

The final test report stores the SHA-256 hash of the checkpoint used for evaluation. During finalization, the stored test-report hash was compared with the current best.pt file and matched exactly. The model was also loaded after cleanup and after folder renumbering to confirm that the final checkpoint remained usable as an Ultralytics segmentation model.

Evidence check	Verified result
best.pt exists and is nonempty	Passed
best.pt loads as a segmentation model	Passed

Evidence check	Verified result
Test report checkpoint hash matches best.pt	Passed
Training results contain 100 epoch rows	Passed
All remaining project scripts compile	Passed
Dataset counts remained unchanged after cleanup	Passed
Script paths were updated after folder renumbering	Passed

10. Final Evaluation Results

10.1 Primary test metrics

Metric	Bounding boxes	Segmentation masks
Precision	99.994%	99.994%
Recall	100.000%	100.000%
mAP@50	99.500%	99.500%
mAP@50-95	91.190%	78.962%

These values come from the final research test split. For instance segmentation, there is no single ordinary accuracy value that replaces all evaluation measures. The clearest headline is 99.5% mAP@50. The stricter mask mAP@50-95 of 78.96% shows that exact mask localization is more difficult than detecting an object at the 0.50 IoU threshold.

10.2 Evaluation speed

The final test evaluation reported approximately 1.0 ms preprocessing, 3.7 ms inference, and 2.7 ms postprocessing per image on the NVIDIA A100 GPU. These timings are hardware-dependent and should not be presented as expected performance on ordinary laptops, CPUs, mobile devices, or embedded systems.

Recommended reporting sentence: The YOLO11m-seg model achieved 99.5% box and mask mAP@50 on the internal research test split, with box mAP@50-95 of 91.19% and mask mAP@50-95 of 78.96%.

11. Final Model, Results, and Log Locations

11.1 Final model checkpoints

File	Purpose	Exact Google Drive path
best.pt	Final checkpoint for prediction, evaluation, export, and demonstration.	/content/drive/MyDrive/Segmentation_Project/04_Training_Runs/banana_segmentation_full_v1_20260805_184146/weights/best.pt
last.pt	Final-epoch checkpoint for recovery or continued training.	/content/drive/MyDrive/Segmentation_Project/04_Training_Runs/banana_segmentation_full_v1_20260805_184146/weights/last.pt

Which model to use: Use best.pt for demonstrations, prediction, validation, export, and deployment testing. It is the checkpoint selected through validation performance.

11.2 Model integrity hashes

Checkpoint	SHA-256
best.pt	cde8ddf49e2c9cdd8a360a3da82822c24c1ddfb36cd3667c572222f37286bba
last.pt	de3437ffe0516dde27bed420ea40891001292e087526657b502772ccada389cc

11.3 Centralized evidence and results

Artifact	Current location
Epoch-by-epoch metrics	/content/drive/MyDrive/Segmentation_Project/07_Results/01_Final_Training/results.csv
Training arguments	/content/drive/MyDrive/Segmentation_Project/07_Results/01_Final_Training/args.yaml
Training curves and plots	/content/drive/MyDrive/Segmentation_Project/07_Results/01_Final_Training/
Final test report	/content/drive/MyDrive/Segmentation_Project/07_Results/02_Final_Test_Evaluation/test_evaluation.json
Final test plots	/content/drive/MyDrive/Segmentation_Project/07_Results/02_Final_Test_Evaluation/
Machine-readable summary	/content/drive/MyDrive/Segmentation_Project/07_Results/final_metrics.json
Final logs and audit reports	/content/drive/MyDrive/Segmentation_Project/05_Logs
Project visualizations	/content/drive/MyDrive/Segmentation_Project/08_Visualizations

12. Using the Trained Model

12.1 Basic prediction in Google Colab

```

from pathlib import Path
import sys

PROJECT_ROOT = Path("/content/drive/MyDrive/Segmentation_Project")
ULTRALYTICS_SOURCE = PROJECT_ROOT / "02_Repositories" / "ultralytics"
BEST_MODEL = PROJECT_ROOT / "04_Training_Runs" / \
    "banana_segmentation_full_v1_20260805_184146" / "weights" / "best.pt"

sys.path.insert(0, str(ULTRALYTICS_SOURCE))

from ultralytics import YOLO

model = YOLO(str(BEST_MODEL))
results = model.predict(
    source="path/to/image_or_folder",
    imgsz=640,
    conf=0.25,
    save=True,
)

for result in results:
    print(result.bboxes)
    print(result.masks)

```

12.2 Re-evaluate the final test split

```

from pathlib import Path
import sys

PROJECT_ROOT = Path("/content/drive/MyDrive/Segmentation_Project")
sys.path.insert(0, str(PROJECT_ROOT / "02_Repositories" / "ultralytics"))

from ultralytics import YOLO

model = YOLO("/content/drive/MyDrive/Segmentation_Project/04_Training_Runs/banana_segmentation_full_v1_20260805_184146/weights/best.pt")
metrics = model.val(
    data="/content/drive/MyDrive/Segmentation_Project/01_Datasets/Prepared_Dataset/research/dataset.yaml",
    split="test",
    imgsz=640,
    batch=16,
    device=0,
    plots=True,
)

print(metrics.results_dict)

```

13. Sharing and Reviewing the Project through Google Drive

The project is stored in Google Drive rather than in a public user-project repository. The complete Segmentation_Project folder can therefore be shared directly with the teacher using Google Drive permissions.

13.1 Recommended sharing steps

1. Open Google Drive and locate the Segmentation_Project folder.
2. Upload README.md to the project root.
3. Create a 09_Documentation folder and upload the updated DOCX and PDF documentation there.
4. Right-click Segmentation_Project and select Share.
5. Add the teacher's email address, or select Anyone with the link when appropriate.
6. Use Viewer permission unless the teacher needs to modify files.
7. Send the shared folder link and identify best.pt as the final model.

13.2 Recommended teacher review order

Order	What to review
1	README.md at the project root
2	09_Documentation/ complete DOCX or PDF
3	06_Scripts/ for every final project script
4	07_Results/ for metrics, curves, and test evidence
5	05_Logs/ for finalization and verification logs
6	04_Training_Runs/.../weights/best.pt for prediction or evaluation
7	01_Datasets/Prepared_Dataset/research/ for independent test evaluation

13.3 Review without changing the original files

A teacher can review scripts, reports, plots, and metadata with Viewer access. To execute the model, the shared project may be copied into the teacher's own Google Drive or downloaded to a compatible environment. The original shared folder should remain read-only during review so the final evidence is not accidentally changed.

14. Reproducing Training and Evaluation

14.1 Required preparation

1. Mount Google Drive in Colab and set PROJECT_ROOT to the shared Segmentation_Project folder.
2. Use the included local Ultralytics source at 02_Repositories/ultralytics, or install the compatible Ultralytics 8.4.115 package.
3. Install a PyTorch build suitable for the available CPU or CUDA environment.
4. Confirm that yolo11m-seg.pt exists in 03_Pretrained_Weights.
5. Confirm that the prepared research dataset and dataset.yaml exist under 01_Datasets.
6. Run environment and dataset verification before training.

14.2 Run final-pipeline training

```
from pathlib import Path
import sys

PROJECT_ROOT = Path("/content/drive/MyDrive/Segmentation_Project")
sys.path.insert(0, str(PROJECT_ROOT / "06_Scripts"))
sys.path.insert(0, str(PROJECT_ROOT / "02_Repositories" / "ultralytics"))

from training_config import TrainingConfig
from trainer import Trainer

config = TrainingConfig(
    experiment_name="banana_segmentation_full_v1",
    dataset="research",
```

```

model="yolo11m-seg",
epochs=100,
batch=16,
imgsz=640,
workers=2,
device=0,
seed=42,
deterministic=True,
amp=True,
cache=False,
optimizer="auto",
patience=50,
save=True,
save_period=10,
plots=True,
pretrained=True,
)

result = Trainer(config).train(sanity=False, resume="auto")
print(result)

```

14.3 Verify the reviewed model

A reviewer should not rely only on screenshots or a manually typed metrics table. They should load best.pt, use the research dataset YAML, run the documented test command, and compare the resulting metrics with final_metrics.json, test_evaluation.json, results.csv, plots, and the stored best.pt SHA-256 hash.

15. Limitations and Responsible Interpretation

- The dataset originated from only eight independent source images.
- Augmentations can remain visually related even when separated by source-family grouping.
- The test split is internal and does not represent every farm, camera, background, lighting condition, banana variety, maturity level, or disease presentation.
- The 99.5% mAP@50 result must not be described as guaranteed 99.5% real-world accuracy.
- The mask mAP@50-95 of 78.96% shows that strict pixel-level localization is more difficult than detection at IoU 0.50.
- The model should be tested on completely new external images before production or broad scientific generalization claims.
- Inference speed measured on an A100 GPU should not be used as a performance estimate for ordinary devices.

16. Final Project Closure

Closure item	Status
Official framework source selected and verified	Completed
Dataset prepared and verified	Completed
Research and benchmark splits created	Completed
Full YOLO11m-seg training completed	Completed
best.pt and last.pt preserved	Completed
Final test evaluation completed	Completed
Results centralized in 07_Results	Completed
Incomplete and duplicate development artifacts removed	Completed
Folder numbering corrected from 01 through 08	Completed
Script path references updated and compiled	Completed
Final documentation and README updated for Huzaifa Waqar Butt	Completed

The project is closed with best.pt as the final model and Huzaifa Waqar Butt as the documented author. Future work should prioritize collecting additional independent source images and conducting external validation rather than relying only on the current augmented dataset.

Appendix A. Key Files and Reports

Artifact	Purpose / location
README.md	Place at the project root for the first review entry point.
Complete DOCX and PDF documentation	Recommended location: 09_Documentation/.
best.pt	/content/drive/MyDrive/Segmentation_Project/04_Training_Runs/banana_segmentation_full_v1_20260805_184146/weights/best.pt
last.pt	/content/drive/MyDrive/Segmentation_Project/04_Training_Runs/banana_segmentation_full_v1_20260805_184146/weights/last.pt
results.csv	/content/drive/MyDrive/Segmentation_Project/07_Results/01_Final_Training/results.csv
args.yaml	/content/drive/MyDrive/Segmentation_Project/07_Results/01_Final_Training/args.yaml
Training plots	/content/drive/MyDrive/Segmentation_Project/07_Results/01_Final_Training/
test_evaluation.json	/content/drive/MyDrive/Segmentation_Project/07_Results/02_Final_Test_Evaluation/test_evaluation.json
final_metrics.json	/content/drive/MyDrive/Segmentation_Project/07_Results/final_metrics.json
Final scripts	/content/drive/MyDrive/Segmentation_Project/06_Scripts
Final logs	/content/drive/MyDrive/Segmentation_Project/05_Logs
Research dataset YAML	/content/drive/MyDrive/Segmentation_Project/01_Datasets/Prepared_Dataset/research/dataset.yaml

Appendix B. Teacher Review Checklist

- Confirm that the author is Huzaifa Waqar Butt.
- Open README.md and verify the final folder structure.
- Inspect the 12 scripts under 06_Scripts.
- Open 07_Results/final_metrics.json and compare it with the documented table.
- Inspect training results.csv and verify that it contains 100 epoch rows.
- Inspect the test-evaluation JSON and plots.
- Calculate or compare the best.pt SHA-256 hash.
- Load best.pt and run prediction on one or more banana images.
- Run test validation when a compatible Colab or CUDA environment is available.
- Review the dataset limitation before interpreting the reported metrics.

References

1. Ultralytics official framework source: <https://github.com/ultralytics/ultralytics>
2. Ultralytics documentation: <https://docs.ultralytics.com>
3. PyTorch documentation: <https://pytorch.org>
4. OpenCV documentation: <https://opencv.org>
5. Ultralytics instance-segmentation dataset format documentation.